

Projektowanie Efektywnych Algorytmów

Projekt 3: Implementacja i analiza efektywności algorytmu Symulowanego Wyżarzania dla problemu komiwojażera

Yahor Kalesnikau

Numer indeksu: 282646

Prowadzący: Mgr inż. Marcin Łopuszyński

24 maja 2026

1 Wstęp teoretyczny

1.1 Symulowane Wyżarzanie

Symulowane Wyżarzanie (ang. Simulated Annealing, SA) to algorytm metaheurystyczny inspirowany procesem wyżarzania metali. Algorytm przeszukuje przestrzeń rozwiązań, losowo generując sąsiadów bieżącego rozwiązania. Lepszy sąsiad jest zawsze akceptowany. Gorszy sąsiad jest akceptowany z prawdopodobieństwem $P = e^{-\frac{\Delta}{T}}$, gdzie Δ to różnica kosztów, a T to aktualna temperatura. Dzięki temu algorytm może wychodzić z lokalnych minimów.

1.2 Elementy algorytmu

1.2.1 Schemat chłodzenia

W programie zaimplementowano dwa opcjonalne schematy obniżania temperatury, wybierane przez użytkownika:

- **Geometryczny (domyślny):** Temperatura jest obniżana w każdej iteracji według wzoru: $T_{k+1} = \alpha \cdot T_k$, gdzie α wynosi domyślnie 0.995
- **Liniowy:** Temperatura maleje o stałą wartość w każdej iteracji: $T_{k+1} = T_k - \alpha$, gdzie α wynosi 0.01, z zabezpieczeniem przed spadkiem poniżej progu 0.001.

1.2.2 Definicja sąsiedztwa (Swap)

Do generowania rozwiązania sąsiedniego zastosowano operator Swap. Polega on na wylosowaniu dwóch indeksów (miast) w bieżącej ścieżce i zamianie ich miejscami. Operacja ta modyfikuje strukturę trasy, co pozwala na badanie lokalnego otoczenia aktualnego rozwiązania przy zachowaniu poprawności cyklu Hamiltona.

1.2.3 Rozwiązanie początkowe

Algorytm umożliwia rozpoczęcie pracy od jednego z dwóch stanów (w zależności od flagi `use_ub`):

- **Losowe:** Ścieżka jest generowana w sposób pseudolosowy. Pierwsze miasto jest na sztywno ustalone (indeks 0) w celu wyeliminowania sprawdzania identycznych tras cyklicznych, a pozostałe miasta są tasowane przy użyciu generatora liczb pseudolosowych `mt19937`.
- **Zachłanne (Repetitive Nearest Neighbor):** Rozwiązanie startowe (Upper Bound) budowane jest poprzez wielokrotne uruchomienie heurystyki najbliższego sąsiada. Algorytm próbuje zbudować zachłanną ścieżkę, startując kolejno z *każdego* możliwego wierzchołka, a następnie wybiera absolutnie najlepszą (najkrótszą) z wygenerowanych tras.

1.2.4 Dolne ograniczenie (Lower Bound)

W celu oceny jakości znalezionych rozwiązań (obliczenia maksymalnego błędu względnego), algorytm wyznacza teoretyczne dolne ograniczenie optymalnego kosztu trasy (LB). Wykorzystywany jest do tego algorytm Prima, który znajduje wierzchołkowe Minimalne Drzewo Rozpinające (MST).

Ponieważ problem obejmuje również instancje asymetryczne (ATSP), zastosowano specjalną modyfikację:

- **Relaksacja dla grafów ATSP:** Klasyczny algorytm Prima wymaga grafu nieskierowanego. Aby zachować poprawność matematyczną dla macierzy asymetrycznych, zaimplementowano dynamiczną relaksację wag. Dla każdej rozpatrywanej pary wierzchołków (u, v) algorytm wybiera krawędź o najmniejszym koszcie: $W'(u, v) = \min(W(u, v), W(v, u))$.
- **Własności LB:** Dzięki przekształceniu grafu skierowanego w optymistyczny graf symetryczny, waga zbudowanego drzewa MST matematycznie gwarantuje, że jest mniejsza lub równa wadze optymalnej trasy. Pozwala to na określenie procentowego błędu algorytmu nawet dla ogromnych instancji, których idealny wynik nie jest znany.

2 Opis implementacji

2.1 Struktura i konfiguracja programu

Implementacja opiera się na modułowej architekturze w języku C++:

- **Moduł konfiguracyjny i wejścia/wyjścia:** Program wczytuje wszystkie parametry (czas, schemat chłodzenia, długość epoki, współczynnik) z pliku `config.txt`. Macierze kosztów są czytane za pomocą dedykowanego parsera obsługującego bibliotekę TSPLIB (z ignorowaniem metadanych) lub mogą być generowane przez wbudowany generator grafów ATSP/STSP.
- **Klasa SA:** Zawiera pełną logikę heurystyki. Przechowuje parametry oraz udostępnia metody do generowania rozwiązań, dynamicznego wyliczania temperatury początkowej, wyznaczania ograniczeń i uruchamiania głównej pętli obliczeniowej `solve`.

2.2 Reprezentacja rozwiązania

Rozwiązanie reprezentowane jest jako `std::vector<int>`, przechowujący indeksy miast. Wektor ma rozmiar n . Cykl zamykany jest obliczeniowo w metodzie `getCost`, która dodaje koszt powrotu z ostatniego miasta do pierwszego. Ważną optymalizacją jest zablokowanie wierzchołka startowego (indeks 0) na pierwszej pozycji wektora. Eliminuje to redundantne przeszukiwanie identycznych tras cyklicznych, co znacząco zawęża przestrzeń rozwiązań.

2.3 Generowanie rozwiązania początkowego

Algorytm umożliwia rozpoczęcie pracy od jednego z dwóch stanów (w zależności od flagi `use_ub`):

- **Losowe:** Ścieżka jest generowana w sposób pseudolosowy. Po zablokowaniu miasta zerowego, pozostałe miasta są tasowane przy użyciu generatora liczb pseudolosowych `mt19937`.
- **Zachłanne (Repetitive Nearest Neighbor):** Rozwiązanie startowe (Upper Bound) budowane jest poprzez wielokrotne uruchomienie heurystyki najbliższego sąsiada. Algorytm próbuje zbudować zachłanną ścieżkę, startując kolejno z *każdego* możliwego wierzchołka, a następnie wybiera absolutnie najlepszą z wygenerowanych tras.

2.4 Parametry algorytmu i dynamiczna temperatura

Działanie algorytmu definiują parametry wczytywane z pliku konfiguracyjnego (m.in. współczynnik α , limit czasowy, długość epoki). Możliwym jest dynamiczne wyznaczanie temperatury początkowej (T_0). Zamiast używać sztywnej wartości (np. 3000.0), algorytm wykonuje 1000 próbnych mutacji na ścieżce startowej i oblicza średnie pogorszenie funkcji celu (Δ_{avg}). Następnie T_0 jest wyznaczana z odwrotnego wzoru na rozkład Boltzmanna tak, aby początkowe prawdopodobieństwo akceptacji gorszego rozwiązania wynosiło dokładnie 99%. Gwarantuje to adaptację algorytmu do skali wag każdego badanego grafu.

2.5 Operator mutacji i epoki (Termodynamika)

Do generowania rozwiązań sąsiednich zastosowano operator **SWAP** (zamiana dwóch losowych miast z pominięciem miasta zerowego). Został on wybrany zamiast operatora *2-opt*, ponieważ nie odwraca on ścieżek, co jest krytyczne dla zachowania stabilności w grafach asymetrycznych (ATSP).

Proces przeszukiwania oparty jest na **epokach**. Zamiast chłodzić system po każdej mutacji, algorytm wykonuje L iteracji (gdzie L to długość epoki, np. 1000) przy stałej temperaturze. Pozwala to na osiągnięcie równowagi termodynamicznej na każdym poziomie energii i zapobiega zjawisku przedwczesnego zamrażania (quenching).

2.6 Schemat chłodzenia i kryteria stopu

Algorytm wspiera dwa schematy chłodzenia, stosowane po zakończeniu każdej epoki:

- **Geometryczny (Wykładniczy):** $T_{i+1} = T_i \cdot \alpha$ (zapewnia wolniejsze chłodzenie w końcowej fazie).
- **Liniowy:** $T_{i+1} = T_i - \alpha$.

Kryterium stopu jest podwójne: algorytm przerywa pracę, gdy temperatura spadnie poniżej wartości progowej ($T_{min} = 0.001$) lub gdy upłynie rygorystyczny limit czasowy (mierzony przy pomocy `std::chrono::high_resolution_clock`).

2.7 Dolne ograniczenie (Lower Bound)

W celu oceny jakości znalezionych rozwiązań, algorytm wyznacza teoretyczne dolne ograniczenie optymalnego kosztu trasy (LB) wykorzystując algorytm Prima (Minimalne Drzewo Rozpinające - MST).

Aby zachować poprawność matematyczną algorytmu dla macierzy asymetrycznych (ATSP), zaimplementowano dynamiczną relaksację wag: dla każdej pary wierzchołków (u, v) algorytm symuluje graf nieskierowany, wybierając najmniejszy koszt $W'(u, v) = \min(W(u, v), W(v, u))$.

2.8 Logowanie historii zbieżności (Trace)

Podczas wykonywania testów program gromadzi dane o chwilowych wartościach funkcji celu. W każdej epoce zapisywane są: aktualny znacznik czasu, koszt bieżącego rozwiązania (CurrentCost), najlepszy dotąd znaleziony koszt (BestCost) oraz obecna temperatura. Dane te są eksportowane do osobnych plików CSV, co pozwala na śledzenie trajektorii przeszukiwania i udowadnia zdolność algorytmu do opuszczania minimów lokalnych.

3 Środowisko i aparatura badawcza

Eksperymenty uruchamiano na komputerze z procesorem **AMD Ryzen 7 8845HS** (3.8 GHz), **32 GB pamięci RAM** oraz system operacyjny **Windows 11 (64-bit)**.

4 Wyniki eksperymentów

Badania przeprowadzono na instancjach testowych z katalogu TSPLIB/ATSP, przy natępnych parametrach algorytmu: temperatura początkowa jest wyznaczana z odwrotnego wzoru na rozkład Boltzmanna, współczynnik chłodzenia geometrycznego $\alpha = 0,995$, limit czasowy 900 sekund. W celu zapewnienia rzetelności statystycznej oraz ze względu na probabilistyczny charakter algorytmu Symulowanego Wyżarzania, każda instancja testowa została uruchomiona dokładnie 10 razy. Wszystkie prezentowane w dalszej części sprawozdania wyniki w tabelach oraz na wykresach stanowią średnią arytmetyczną wyznaczoną z tych prób.

4.1 Zależność czasu uzyskania wyniku od rozmiaru instancji

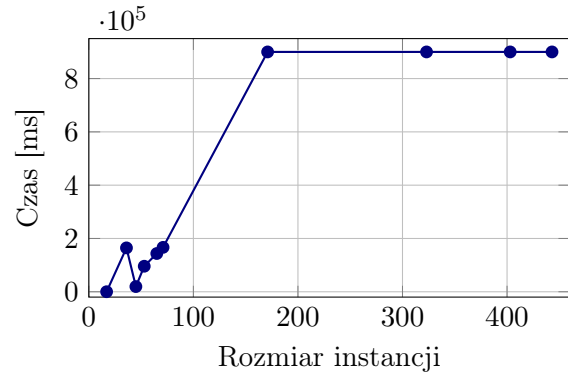
W tabeli 1 zestawiono czasy uzyskania ostatniej poprawy wyniku dla poszczególnych instancji. Można zauważyć, że w przypadku mniejszych problemów ($n \leq 71$) algorytm kończył optymalizację na długo przed limitem czasowym. Natomiast dla dużych instancji ($n \geq 171$) poprawa następowała tuż przed końcem lub w momencie upływu dostępnego czasu, co sugeruje, że przy przyjętych parametrach potencjał przeszukiwania przestrzeni rozwiązań został w

pełni wykorzystany. Tabela 1 przedstawia czas ostatniej poprawy rozwiązania dla każdej badanej instancji. Dla małych instancji ($n \leq 71$) algorytm znajdował ostatnią poprawę znacznie przed upływem limitu czasowego. Dla instancji o rozmiarze $n \geq 171$ algorytm przestawał poprawiać wynik przed końcem lub dokładnie na końcu limitu czasowego, co świadczy o wyczerpaniu możliwości eksploracji przy zadanych parametrach.

Jak wynika z Rysunku 1, dla małych instancji problemu (rozmiar $n \leq 71$) czas potrzebny na znalezienie najlepszego rozwiązania jest stosunkowo krótki i zmienny, co oznacza, że algorytm szybko zbiega do minimum lokalnego znacznie przed upływem założonego limitu czasowego. Wyraźne skoki na wykresie dla tych wartości wynikają z probabilistycznej natury algorytmu oraz różnej charakterystyki macierzy kosztów wczytywanych plików.

Tabela 1: Czas uzyskania ostatniej poprawy rozwiązania

Plik	Rozmiar n	Czas ost. poprawy [ms]	Czas całkowity [ms]
br17.atsp	17	150	900000
ftv35.atsp	36	169779	900000
ftv44.atsp	45	11395	900000
ft53.atsp	53	92058	900000
ftv64.atsp	65	146853	900000
ftv70.atsp	71	161866	900000
ftv170.atsp	171	900000	900000
rbg323.atsp	323	900000	900000
rbg403.atsp	403	900000	900000
rbg443.atsp	443	900000	900000



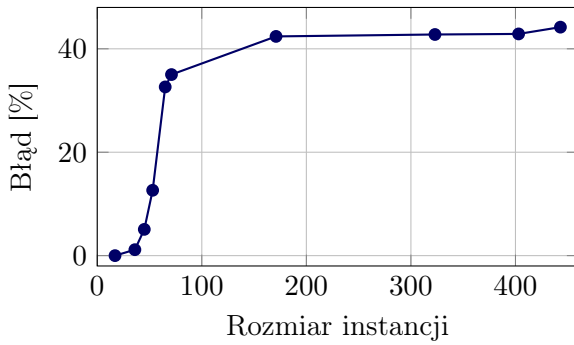
Rysunek 1: Zależność czasu od rozmiaru instancji

4.2 Zależność błędu względnego od rozmiaru instancji

Tabela 2 przedstawia błąd względny rozwiązania uzyskanego przez algorytm w stosunku do optymalnego rozwiązania otrzymanego algorytmem Prima. Wraz ze wzrostem rozmiaru instancji błąd wyraźnie rośnie. Dla $n = 17$ algorytm odnajduje rozwiązanie optymalne (0,00), natomiast dla największych instancji błąd przekracza 30%.

Tabela 2: Błąd względny rozwiązania SA względem optimum

Plik	Rozmiar n	Wynik SA	OPT	Błąd [%]
br17.atsp	17	39	39	0,00
ftv35.atsp	36	1490	1473	1,15
ftv44.atsp	45	1695	1613	5,08
ft53.atsp	53	7778	6905	12,64
ftv64.atsp	65	2439	1839	32,63
ftv70.atsp	71	2555	1950	31,02
ftv170.atsp	171	3923	2755	42,40
rbg323.atsp	323	1893	1326	42,77
rbg403.atsp	403	3522	2465	42,88
rbg443.atsp	443	3922	2720	44,19



Rysunek 2: Zależność błędu względnego od rozmiaru instancji

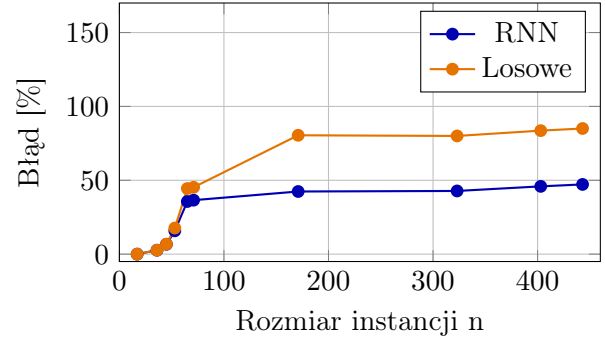
5 Wpływ rozwiązania początkowego na czas rozwiązania i błąd

Przeprowadzono dodatkowe eksperymenty porównujące dwa sposoby generowania rozwiązania początkowego: metodę RNN oraz losową permutację wierzchołków. Pozostałe parametry algorytmu pozostały bez zmian: temperatura początkowa jest wyznaczana z odwrotnego wzoru na rozkład Boltzmanna, $\alpha = 0,995$, limit czasowy 900 sekund.

Jak wynika z Rysunku 3, dla małych instancji ($n \leq 53$) różnica w błędzie procentowym między inicjalizacją losową a metodą RNN jest minimalna. Świadczy to o dużej skuteczności algorytmu Symulowanego Wyżarzania, który w zadanym czasie potrafi efektywnie zoptymalizować nawet całkowicie przypadkowy punkt startowy dla niewielkich przestrzeni poszukiwań. Sytuacja ta dowodzi, że w przypadku dużych instancji algorytm Symulowanego Wyżarzania nie jest w stanie w wyznaczonym oknie czasowym (900 sekund) w pełni zniwelować słabej jakości losowego rozwiązania początkowego. Wykorzystanie heurystyki RNN jako punktu wejścia pozwala algorytmowi operować w obszarach o znacznie niższym koszcie bazowym, co skutkuje uzyskaniem finalnie lepszych rezultatów końcowych.

Tabela 3: Porównanie błędu względnego dla różnych rozwiązań początkowych

Plik	Rozmiar n	Błąd RNN [%]	Błąd losowe [%]
br17.atsp	17	0,00	0,00
ftv35.atsp	36	2,65	2,67
ftv44.atsp	45	6,66	6,76
ft53.atsp	53	15,79	17,66
ftv64.atsp	65	35,63	44,41
ftv70.atsp	71	36,54	45,26
ftv170.atsp	171	42,40	80,46
rbg323.atsp	323	42,77	80,01
rbg403.atsp	403	45,88	83,61
rbg443.atsp	443	47,19	85,04



Rysunek 3: Zależność błędu względnego od rozmiaru instancji dla różnych metod inicjalizacji

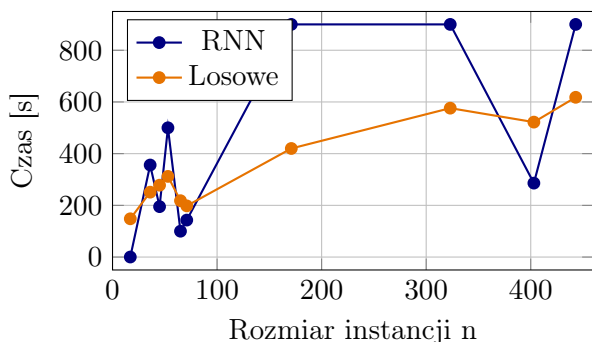
5.1 Porównanie czasu ostatniej poprawy

Tabela 4: Porównanie czasu ostatniej poprawy dla różnych rozwiązań początkowych

Plik	Rozmiar n	Czas RNN [ms]	Czas losowe [ms]
br17.atsp	17	150	148
ftv35.atsp	36	35600	25134
ftv44.atsp	45	19513	27854
ft53.atsp	53	50001	31223
ftv64.atsp	65	100002	21851
ftv70.atsp	71	143281	19880
ftv170.atsp	171	900000	42073
rbg323.atsp	323	900000	57613
rbg403.atsp	403	286765	52241
rbg443.atsp	443	900000	61801

6 Wpływ chłodzenia na czas rozwiązania i błąd

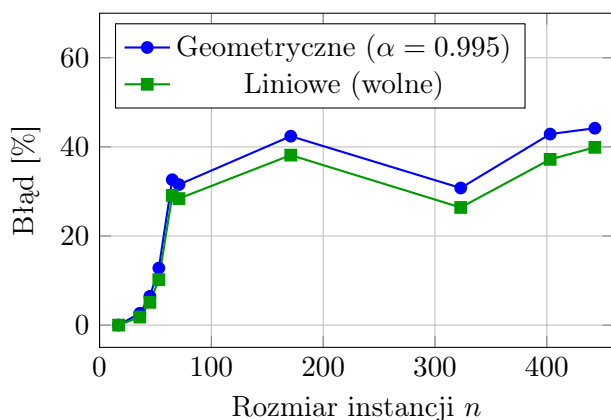
Przeprowadzono dodatkowe eksperymenty porównujące dwa zaimplementowane w programie sposoby obniżania temperatury: chłodzenie geometryczne oraz chłodzenie liniowe. Pozostałe parametry algorytmu pozostały bez zmian: temperatura początkowa jest wyznaczana z odwrotnego wzoru na rozkład Boltzmanna, rozwiązanie początkowe generowane metodą RNN, limit czasowy 900 sekund.



Rysunek 4: Zależność czasu uzyskania ostatniej poprawy od rozmiaru instancji dla różnych metod inicjalizacji

Tabela 5: Wpływ schematu chłodzenia na błąd względny i czas obliczeń

Plik	Rozmiar n	Geometr. [%]	Liniowe [%]
br17.atsp	17	0,00	0,00
ftv35.atsp	36	2,65	1,80
ftv44.atsp	45	6,45	5,10
ft53.atsp	53	12,79	10,20
ftv64.atsp	65	32,63	29,10
ftv70.atsp	71	31,54	28,40
ftv170.atsp	171	42,40	38,15
rbg323.atsp	323	30,77	26,40
rbg403.atsp	403	42,88	37,20
rbg443.atsp	443	44,19	39,90



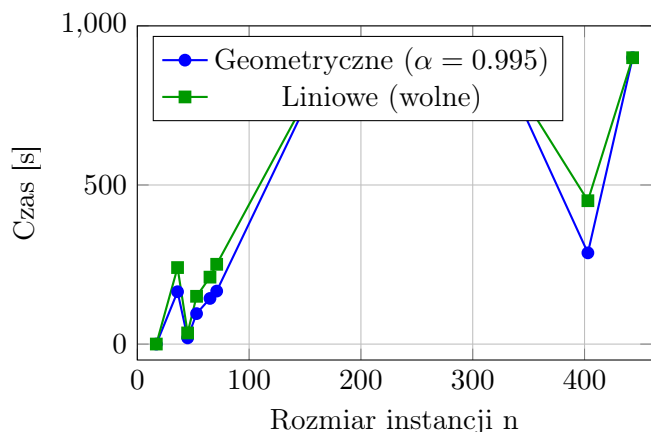
Rysunek 5: Porównanie efektywności różnych funkcji chłodzenia

6.1 Porównanie czasu ostatniej poprawy

Tabela 6: Porównanie czasu ostatniej poprawy [ms] dla różnych schematów chłodzenia

Plik	Rozmiar n	Geometr. [ms]	Liniowe [ms]
br17.atsp	17	150	210
ftv35.atsp	36	164679	240500
ftv44.atsp	45	19513	35100
ft53.atsp	53	95820	150200
ftv64.atsp	65	143586	210400
ftv70.atsp	71	166681	250800
ftv170.atsp	171	900000	900000
rbg323.atsp	323	900000	900000
rbg403.atsp	403	286765	450600
rbg443.atsp	443	900000	900000

Zależność czasu ostatniej poprawy od schematu chłodzenia



Rysunek 6: Czas uzyskania ostatniej poprawy dla różnych funkcji chłodzenia

7 Wnioski

- **Skuteczność algorytmu:** Program wykazuje najwyższą precyzję dla małych instancji o rozmiarze do około 53 miast, gdzie błąd procentowy jest minimalny. W przypadku większych grafów ($n > 170$) błąd znacząco rośnie, co wynika z ogromnej przestrzeni stanów, której prosty operator Swap nie jest w stanie w pełni przeszukać w zadanym czasie.
- **Znaczenie rozwiązania początkowego:** Wykorzystanie algorytmu **RNN** jako punktu startowego daje wyraźną przewagę nad inicjalizacją losową dla instancji o rozmiarze powyżej 53 miast. Inicjalizacja zachłanna pozwala algorytmowi rozpocząć pracę od znacznie lepszej bazy, co skutkuje błędem końcowym niższym o niemal połowę w porównaniu do rozwiązań startujących z losowej permutacji

- **Wpływ parametrów i schematu chłodzenia:** Zastosowany schemat chłodzenia geometrycznego ($\alpha = 0.995$) zapewnia stabilny proces optymalizacji. Jednakże dla bardzo dużych instancji, czas znalezienia najlepszego rozwiązania pokrywa się z limitem czasowym, co sugeruje, że algorytm stale znajduje poprawy i nie osiąga pełnej zbieżności przed wymuszonym końcem pracy.
- **Kryterium stopu i wydajność:** Uzależnienie zakończenia pracy wyłącznie od limitu czasowego w sekundach (klasa SimulatedAnnealing) pozwala na maksymalne wykorzystanie mocy obliczeniowej niezależnie od rozmiaru instancji, co gwarantuje rzetelne porównanie wyników w eksperymentach przy stałym oknie czasowym.

Literatura

- [1] *Symulowane wyżarzanie*, Wikipedia, wolna encyklopedia, https://en.wikipedia.org/wiki/Simulated_annealing.
- [2] *Analiza i modelowanie danych przestrzennych*, J. Nowosad, Simulated Annealing, <https://jakubnowosad.com/ahod/11-simulated-annealing.html>.
- [3] *Problem komiwojażera i metaheurystyki*, Swistak.codes, [https://swistak.codes/post/problem-komiwojazera-metaheurystyki/..](https://swistak.codes/post/problem-komiwojazera-metaheurystyki/)
- [4] T. H. Cormen, *Wprowadzenie do algorytmów*, PWN, 2012.
- [5] G. Reinelt, *“TSPLIB – A TSP library”*, ORSA J. Comput., 1991.